

Shellcode

版权所有 © 2022 杜文亮,保留所有权利。
允许个人使用。如需在课堂上使用这些习题,仅当该课程采用作者的著作作为教材时方可允许。所有其他用途均须事先征得作者的同意。

S9.1. shellcode (32 位)的本质是在调用 execve() 系统调用之前准备好四个寄存器:eax、ebx、ecx 和 edx。请描述这四个寄存器应该包含哪些值。

S9.2. 在基于栈的方法中,我们需要将命令字符串存储在内存中,然后将该字符串的地址保存到 ebx 中。请编写一段代码 (32 位),将字符串 “aaaabbbbccccddd”存储在内存中,然后将其地址保存到 ebx 中。

S9.3. 在基于栈的方法中,我们需要将参数数组 argv[] 存储在内存中,然后将该数组的地址存储在 ecx 中。请编写一段代码 (32 位)在内存中构建以下 argv[] 数组,然后将其地址赋值给 ecx。

```
ecx。

argv[0] = 0x11111111 argv[1] =
0x22222222 argv[2] = 0x33333333
argv[3] = 0x00000000
```

S9.4. 与栈式方法相比,代码段应用的主要区别是什么?
编写 shellcode 的方法是什么?

S9.5. 以下 shellcode 不完整。你需要将所有 * 替换为实际的数字。此外,请为每行标有带圆圈的数字添加简短注释,解释其用途。你不能仅仅描述每条指令的含义 (例如,不能说 “pop eax”是从栈中取出一个值并将其存储到 eax 中)。你需要解释其目的,即为什么需要这条指令。

```
节.文本
    全局 _start _start:

        BITS 32
        jmp 短二

    一:
        流行音乐
        xor eax,eax
        mov [ebx+*], al    mov [ebx+*], ebx    mov
[ebx+*], eax    lea ecx, [ebx+*] xor edx, edx
        mov al, 0x0b int 0x80

    二:
        调用数据库 “/bin/
shabcde”
```

```
db  AAAA
    数据库 “BBBB”
```

S9.6. 请将以下32位shellcode中的问号和星号分别替换为数字和寄存器名称。每个问号代表一个数字,每个星号代表一个寄存器名称。请为每行未完成的代码添加注释。

```
节文本
全局_开始
_开始:
    BITS 32
    jmp 短二
一:
    流行音乐 edx
    xor eax, eax mov [*+?], al
    lea ebx, [*+?] mov [*+?], eax
    mov [*+?], ebx

    mov ecx, *
    xor *, * mov al,
    0x0b int 0x80

二:
    打电话给一个人
    db  AAAA
    数据库 “BBBB”
    db  /bin/sh*
```

S9.7. 以下 shellcode 不完整。它的目标是执行以下命令: “/bin/rm -rf *” (不含引号) 。代码已给出部分,并附有一些有用的信息。请完成这段代码。你的代码必须有详细的注释,否则会被扣分。

```
_开始:
    jmp 短二
一:
    ...在此处添加代码...

    mov al, 0x0b ; 调用 execve() 系统调用 int 0x80

二:
    打电话给一个人
    db  abcd/bin/rmab-rf****
    db  AAAABBBBCCCCDDDEEEEEFFFFGGGG
```

S9.8. 为什么 shellcode 通常不允许代码中包含零?

S9.9. 请列举三种去除 shellcode 中零的典型方法。

S9.10. 我们想在栈上存储字符串“ab”,但代码中不允许包含任何零（字符串末尾是二进制零）。(1)请完成小端字节序机器的代码。(2)请完成大端字节序机器的代码。

```
mov ecx, ab** ... (缺少代  
码) ... push ecx
```

S9.11. ★ 请

在 `eax` 寄存器中存储 `0xAA00BB00`。最终机器代码中不能有任何二进制零。